

Machine Learning II

Lecture 5: Embeddings and Text as Data

Alexander Quispe
ENEI – INEI · PEU-CD 2026

October 19, 2026

Outline

Recap and Motivation

Manifold Learning, Briefly

Distance vs Inner-Product Embeddings

Word Embeddings: word2vec

Embeddings for Economics

Deep Embeddings

Static vs Contextual Embeddings

Summary and Next Steps

Recap & Motivation

From latent factors to learned representations

Where We Left Off

Lecture 6 gave us three viewpoints on the *same* object:

1. **SVD / PCA** on a dense matrix $\mathbf{X}$.
2. **Latent factor model** $\mathbf{Y} \approx \mathbf{UV}^\top$ with missing entries.
3. **Panel data** with interactive fixed effects (Bai 2009, Athey et al. 2021).

The common thread: each row of $\mathbf{U}$ is a *vector representation* of a user / unit / entity, and similarity is the dot product $\mathbf{u}_i^\top \mathbf{v}_j$.

Today: these row vectors have a name – they are **embeddings** – and we generalize them well beyond the linear case.

What Is an Embedding?

Definition. An embedding is a map

$$s \longmapsto \mathbf{u}_s \in \mathbb{R}^K$$

from a discrete object s (a word, song, user, country, firm) to a real-valued vector $\mathbf{u}_s$.

Two ways to encode meaning:

- **Distance:** similar objects have similar vectors, $d(\mathbf{u}_s, \mathbf{u}_{s'}) = \|\mathbf{u}_s - \mathbf{u}_{s'}\|^2$.
- **Inner product:** $d(\mathbf{u}_s, \mathbf{u}_{s'}) = \mathbf{u}_s^\top \mathbf{u}_{s'}$.

Where do the vectors come from? We **learn** them from data, by writing down a loss function in which similar objects *should* have similar vectors and letting gradient descent do the work.

The embeddings are not specified by the user – they emerge from the training signal.

Embedding Examples

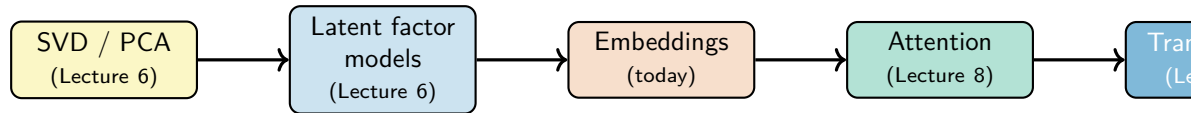
The same mathematical object, applied to different domains:

Domain	Object	Embedding
NLP	a word, e.g. <i>king</i>	$\mathbf{v}_{\text{king}} \in \mathbb{R}^{300}$
Recommenders	user i , movie j	$\mathbf{u}_i, \mathbf{v}_j \in \mathbb{R}^K$
Vision	an image	$\phi(\text{image}) \in \mathbb{R}^{128}$
Multimodal	image + caption	shared $\mathbb{R}^{512}$ (CLIP)
Knowledge graphs	entity + relation	$\mathbb{R}^K$ per entity
Econ panels (L6)	unit i at time t	$\lambda_i, \mathbf{F}_t \in \mathbb{R}^K$

Takeaway

“Good embeddings turn abstract similarity into geometry.”

The Narrative Arc



Reading the arc:

- **Yesterday:** linear factor models on observed matrices.
- **Today:** learn vector representations of *anything* from co-occurrence data.
- **Next week:** make those representations *context-dependent* (attention).

Each step generalizes the previous one. Same idea, growing expressiveness.

Why a Whole Lecture on Embeddings?

Embeddings are everywhere in modern ML:

- Every recommender system: user $\times$ item embeddings.
- Every NLP model: word, subword, sentence embeddings.
- Every retrieval system: query $\times$ document embeddings.
- Every multimodal model: image $\times$ text embeddings (CLIP).
- Every knowledge graph: entity $\times$ relation embeddings.

And especially for economics:

- Text-as-data: embed FOMC speeches, central bank minutes, news.
- Judge / firm / country embeddings from co-occurrence in text or trade.
- Embeddings as features for causal inference.

If you understand embeddings, you understand the basic unit of every modern AI system.

Today's Roadmap (Annotated)

1. **Manifold learning, briefly.** LLE, t-SNE, UMAP – dimensionality reduction as a warm-up.
2. **Distance vs inner-product embeddings.** Two ways to encode similarity; playlist embedding as a worked example.
3. **Word embeddings: word2vec.** Skip-gram, negative sampling, the Levy-Goldberg theorem, classic analogies.
4. **Embeddings for economics.** Text-as-data, judge embeddings, firm embeddings, trade.
5. **Deep embeddings.** Siamese networks, triplet loss, modern contrastive learning.
6. **Static vs contextual.** The limit of word2vec – and the bridge to Transformers.

Manifold Learning, Briefly

Or: why we need t-SNE and UMAP

The Manifold Hypothesis

Claim: high-dimensional data usually lies on (or near) a much lower-dimensional manifold inside the ambient space.

Examples:

- Faces: each 50,000-pixel image is on a ~ 50 -dim manifold (pose, expression, lighting).
- Text: each one-hot $|V|$ -dim word vector lies on a ~ 300 -dim “meaning” manifold.
- Macro time series: a 200-country panel of indicators lies on a low-rank factor manifold.

Goal of manifold learning: find coordinates on that low-dim manifold.

This is a generalization of PCA: instead of a linear subspace, allow the surface to bend.

Locally Linear Embedding: Overview

LLE (Roweis & Saul, *Science* 2000): the data is *locally* linear even if globally curved.

Two-step recipe:

1. For each point $\mathbf{x}_i$, find its B nearest neighbors $\mathcal{B}(i)$, and write $\mathbf{x}_i$ as a convex combination of those neighbors. This produces a weight matrix $\mathbf{W}$ that captures *local geometry*.
2. Find low-dim points $\mathbf{u}_i$ that satisfy the *same* reconstruction weights.

Local reconstruction

Key idea: learn $\mathbf{W}$ from high-dim data, then transfer it to low dim.

We now unpack each of the two steps in detail.

LLE Step 1: Local Reconstruction Weights

Given: n high-dim points $\mathbf{x}_1, \dots, \mathbf{x}_n$ and neighborhoods $\mathcal{B}(i)$.

For each point $\mathbf{x}_i$, solve a small local problem:

$$\min_{W_{i,*}} \left\| \mathbf{x}_i - \sum_{j \in \mathcal{B}(i)} W_{ij} \mathbf{x}_j \right\|^2 \quad \text{s.t.} \quad \sum_{j \in \mathcal{B}(i)} W_{ij} = 1.$$

Concrete example. If $\mathcal{B}(i) = \{3, 17, 22\}$, then

$$\mathbf{x}_i \approx W_{i,3} \mathbf{x}_3 + W_{i,17} \mathbf{x}_{17} + W_{i,22} \mathbf{x}_{22}$$

with $W_{i,3} + W_{i,17} + W_{i,22} = 1$.

Storage. The full weight matrix $\mathbf{W}$ is $n \times n$ but **sparse** – row i has at most B nonzero entries.

If $n = 100$ and $B = 5$, we solve 100 small problems, each with 5 unknowns.

LLE Step 1: The Algebraic Trick

Use the constraint $\sum_j W_{ij} = 1$ to rewrite the residual:

$$\mathbf{x}_i - \sum_j W_{ij} \mathbf{x}_j = \sum_j W_{ij} (\mathbf{x}_i - \mathbf{x}_j).$$

Squared norm expands as a quadratic form:

$$\left\| \sum_j W_{ij} (\mathbf{x}_i - \mathbf{x}_j) \right\|^2 = \sum_{j,k} W_{ij} W_{ik} C_{jk}^i$$

where

$$C_{jk}^i = (\mathbf{x}_i - \mathbf{x}_j)^\top (\mathbf{x}_i - \mathbf{x}_k).$$

Interpretation. C^i is the *local Gram matrix* (covariance of neighbor offsets around $\mathbf{x}_i$).

Closed form. Each row $W_{i,*}$ is found by solving a small linear system with C^i on the left-hand side – B variables, one Lagrange multiplier for the sum-to-one constraint. No iteration needed.

LLE Step 2: Learn the Low-Dim Embedding

Keep $\mathbf{W}$ fixed. Now search for low-dim coordinates $\mathbf{u}_i \in \mathbb{R}^K$ that satisfy the *same* reconstruction:

$$\arg \min_{\mathbf{U}} \sum_i \left\| \mathbf{u}_i - \sum_{j \in \mathcal{B}(i)} W_{ij} \mathbf{u}_j \right\|^2.$$

Interpretation. “If $\mathbf{x}_i$ was a convex combination of its neighbors in high dim, the same convex combination should reproduce $\mathbf{u}_i$ in low dim.” This preserves *local geometry*.

Trivial solution issue. Without constraints, $\mathbf{U} = 0$ minimizes the objective. So we add:

$$\sum_i \mathbf{u}_i = 0 \quad (\text{centering}), \quad \frac{1}{n} \mathbf{U}^\top \mathbf{U} = \mathbf{I}_K \quad (\text{unit variance per axis}).$$

Notation convention used here: $\mathbf{U} \in \mathbb{R}^{n \times K}$ with one row per point.

LLE Step 2: A Small Numerical Example

Setup. Three collinear points in 2D, project to $K = 1$:

$$\mathbf{x}_1 = (0, 0), \quad \mathbf{x}_2 = (1, 1), \quad \mathbf{x}_3 = (2, 2).$$

Step 1 (already done). For $\mathbf{x}_2$ with neighbors $\{\mathbf{x}_1, \mathbf{x}_3\}$, solving the local problem gives

$$W_{2,1} = 0.5, \quad W_{2,3} = 0.5 \quad (\text{since } \mathbf{x}_2 = 0.5 \mathbf{x}_1 + 0.5 \mathbf{x}_3).$$

Step 2. Find scalars $u_1, u_2, u_3 \in \mathbb{R}$ that satisfy the *same* reconstruction:

$$u_2 = 0.5 u_1 + 0.5 u_3.$$

Apply the constraints.

Centering: $u_1 + u_2 + u_3 = 0$. Substituting $u_2 = 0.5(u_1 + u_3)$:

$$1.5(u_1 + u_3) = 0 \Rightarrow u_3 = -u_1, \quad u_2 = 0.$$

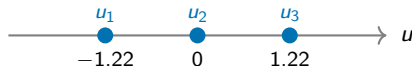
Unit variance: $\frac{1}{3}(u_1^2 + u_2^2 + u_3^2) = 1$. With $u_3 = -u_1$ and $u_2 = 0$:

$$\frac{2u_1^2}{3} = 1 \Rightarrow u_1 = -\sqrt{1.5} \approx -1.22.$$

LLE Step 2: Interpretation of the Numerical Example

Recovered 1D embedding:

$$(u_1, u_2, u_3) = (-\sqrt{1.5}, 0, \sqrt{1.5}) \approx (-1.22, 0, 1.22).$$



What the algorithm achieved:

- The three points lie on a 1D line in 2D; LLE found that line.
- The *ordering* along the line is preserved: $u_1 < u_2 < u_3$.
- The *local reconstruction* holds: $u_2 = 0.5 u_1 + 0.5 u_3 = 0$.
- The sign is arbitrary – $(+\sqrt{1.5}, 0, -\sqrt{1.5})$ is equally valid.

Same algorithm, scaled up. For thousands of points, we solve this via an eigendecomposition (next slide) instead of hand-calculating constraints.

LLE Step 2: Matrix Form and Eigendecomposition

Define $\mathbf{A} = \mathbf{I}_n - \mathbf{W}$. The objective becomes:

$$\sum_i \left\| \mathbf{u}_i - \sum_j W_{ij} \mathbf{u}_j \right\|^2 = \|\mathbf{A}\mathbf{U}\|_{\text{Fro}}^2 = \text{trace}(\mathbf{U}^\top \mathbf{A}^\top \mathbf{A} \mathbf{U}).$$

Let $\mathbf{M} = \mathbf{A}^\top \mathbf{A} = (\mathbf{I}_n - \mathbf{W})^\top (\mathbf{I}_n - \mathbf{W})$.

Entrywise:

$$M_{ij} = \mathbf{1}_{[i=j]} - W_{ij} - W_{ji} + \sum_k W_{ki} W_{kj}.$$

Properties of $\mathbf{M}$:

- Symmetric: $\mathbf{M} = \mathbf{M}^\top$.
- Positive semidefinite: $\mathbf{z}^\top \mathbf{M} \mathbf{z} = \|\mathbf{A} \mathbf{z}\|^2 \geq 0$ for any $\mathbf{z}$.

Solution (min-max theorem applied to $\text{trace}(\mathbf{U}^\top \mathbf{M} \mathbf{U})$ s.t. orthonormality): $\mathbf{U}$ = bottom K *non-trivial* eigenvectors of $\mathbf{M}$ (skipping the zero eigenvalue from $\mathbf{M}\mathbf{1} = 0$).

$\mathbf{M}$ penalizes low-dim embeddings that break the local reconstruction encoded by $\mathbf{W}$.

Section Summary

Manifold learning: find low-dim coordinates that preserve geometric structure.

LLE (Roweis & Saul, *Science* 2000) is the canonical example:

- **Step 1.** Learn local weights $\mathbf{W}$ that reconstruct each point from its neighbors.
- **Step 2.** Find low-dim points $\mathbf{U}$ that satisfy the *same* reconstruction.

Solution: bottom K non-trivial eigenvectors of $\mathbf{M} = (\mathbf{I} - \mathbf{W})^\top (\mathbf{I} - \mathbf{W})$.

Related modern tools (t-SNE, UMAP) follow the same template – preserve a notion of locality between high- and low-dim representations – but use different loss functions.

Critical caveat: manifold-learning outputs are **visualizations**. The 2D pictures they produce should never be used as features for a downstream model.

Coming up: the embeddings we *do* use as features are learned from a different objective – not preserving distances, but predicting co-occurrence in sequences.

Distance vs Inner-Product Embeddings

Two geometries, two interpretations

Two Ways to Encode Similarity

Distance-based:

$$P(s \mid s') \propto \exp(-\|\mathbf{u}_s - \mathbf{u}_{s'}\|^2)$$

Two objects are similar if their vectors are close in Euclidean distance.

Inner-product based:

$$P(s \mid s') \propto \exp(\mathbf{u}_s^\top \mathbf{v}_{s'})$$

Two objects are similar if their vectors point in similar directions.

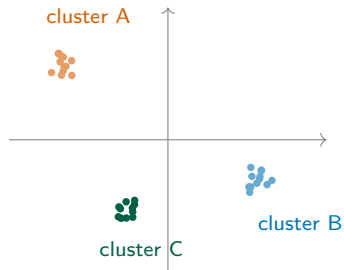
They produce very different geometries:

- Distance $\rightarrow$ clustering: objects of the same “type” form tight clouds.
- Inner product $\rightarrow$ direction: meaning is encoded by orientation, magnitude can vary.

Inner product is exactly the latent factor model from Lecture 6.

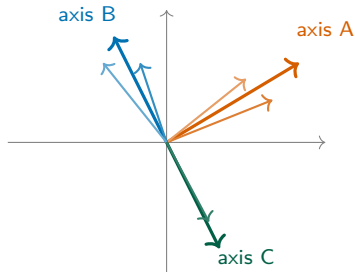
Two Geometries, Side by Side

Distance-based



Clusters in space.

Inner-product based



Directions, varying magnitudes.

Use distance when: objects have a natural notion of locality (songs near other songs of similar genre).

Use inner product when: you want *compositionality* – $\mathbf{u}_a + \mathbf{u}_b \approx \mathbf{u}_c$ makes sense.

Inner Product = Latent Factor Model

Recall from Lecture 6:

$$Y_{ij} \approx \mathbf{u}_i^\top \mathbf{v}_j$$

where $\mathbf{u}_i$ is a user vector and $\mathbf{v}_j$ is an item vector.

Embedding view: $\mathbf{U}$ contains one row per user, $\mathbf{V}$ contains one row per item. The dot product gives a similarity score.

To convert to a probabilistic model:

$$P(\text{item } s \mid \text{user } s') = \frac{\exp(\mathbf{u}_s^\top \mathbf{v}_{s'})}{\sum_{s''} \exp(\mathbf{u}_{s''}^\top \mathbf{v}_{s'})}$$

The softmax form lets us train via maximum likelihood instead of squared error.

Same vectors, different loss. The geometry is shaped by what we minimize.

Worked Example: Playlist Embeddings

Setup (Chen et al., 2012): Spotify-style playlist data, sequences of songs.

Model: for each song s , an entry vector $\mathbf{u}_s$ and an exit vector $\mathbf{v}_s$.

$$P(s \mid s') = \frac{\exp(-\|\mathbf{u}_s - \mathbf{v}_{s'}\|^2)}{\sum_{s''} \exp(-\|\mathbf{u}_{s''} - \mathbf{v}_{s'}\|^2)}$$

Parameters: $2K|S|$ – linear in vocabulary, not quadratic.

Training: maximize the log-likelihood of observed playlists,

$$\arg \max_{\mathbf{U}, \mathbf{V}} \sum_{\text{playlists}} \sum_j \log P(p_i^j \mid p_i^{j-1}).$$

Simpler “single-point” variant: use $\mathbf{u}_s$ for both roles. Less expressive but easier to visualize.

Same mathematical structure will reappear in word2vec.

The Cold-Start Problem

What happens when a new song / item arrives?

- No co-occurrence data yet, so no signal to learn $\mathbf{u}_s$.
- Default to $\mathbf{u}_s = 0$ or random: both are bad.

Solution: side information. If the new song has tags (genre, artist, year), learn an embedding of tags as well:

$$\mathbf{u}_s \approx \frac{1}{|T_s|} \sum_{t \in T_s} \mathbf{a}_t$$

where $\mathbf{a}_t$ is the tag embedding.

Generalization (key idea for what's coming):

- Without side info $\rightarrow$ embedding looked up by ID (matrix factorization).
- With side info $\rightarrow$ embedding *computed* from features.

The cold-start fix is the seed of every two-tower retrieval and deep embedding model.

Word Embeddings: word2vec

The canonical embedding

From Geometric to Sequence-Based Embeddings

Where we were:

- **LLE** (and t-SNE / UMAP): preserve *local geometry*. The training signal is “which points were close in high dim?”

Where we're going:

- **word2vec**: preserve *co-occurrence in sequences*. The training signal is “which words appeared next to each other?”

Same machinery, different signal. Both produce vectors; only the training data differs.

Concrete example. Consider three sentences:

- “the dog jumped over the fence”
- “my dog ate my homework”
- “I walked my dog up to the fence”

The word *dog* appears near *jumped*, *ate*, *walked*, *fence*, *the*, *my*, *over*.

These context co-occurrences are the training signal for $\mathbf{v}_{\text{dog}}$.

Distributional semantics: “a word is known by the company it keeps.”

The Distributional Hypothesis

Linguistic principle

"You shall know a word by the company it keeps."

– J. R. Firth (1957)

Implication: we can learn the meaning of a word from the distribution of words around it.

Example. Consider:

- "The Federal Reserve *raised* interest rates."
- "The central bank *lifted* its policy rate."
- "The Fed *hiked* the benchmark."

Raised, lifted, hiked appear in very similar contexts → they should have similar embeddings.

Word2vec operationalizes this hypothesis as a learning objective.

Why Probability Tables Don't Work

Naive model: a giant table of bigram probabilities $P(w \mid w')$.

For a vocabulary of $|V|$ words:

$ V $	# table entries $ V ^2$
10,000 (small)	10^8
100,000 (typical English)	10^{10}
1,000,000 (Spanish + named entities)	10^{12}

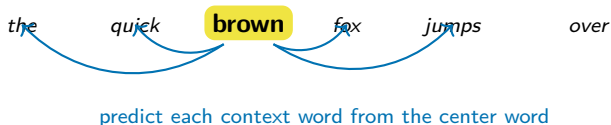
And it gets worse: most (w, w') pairs are never observed, so the model can't generalize.

Embedding solution: represent each word as a K -dim vector with $K \sim 100$. Total parameters: $K|V|$ – linear, not quadratic.

Generalization comes from shared structure: words with similar embeddings transfer information.

Skip-Gram Model: The Objective

Skip-gram (Mikolov, Sutskever, Chen, Corrado, Dean, NIPS 2013): for each word in the corpus, predict the words in its context.



Notation conventions for this section:

- Vocabulary $S = \{s_1, \dots, s_W\}$, size W . Embedding dimension D .
- Corpus is a sequence $w_1, w_2, \dots, w_T$ of tokens from S .
- w_I = center / input word. w_O = output / context word.
- $\mathbf{v}_w \in \mathbb{R}^D$ = **input** embedding (when w is the center).
- $\mathbf{v}'_w \in \mathbb{R}^D$ = **output** embedding (when w is the context).

Objective for window size s over corpus length T :

$$\arg \max_{\mathbf{V}, \mathbf{V}'} \frac{1}{T} \sum_{t=1}^T \sum_{s \leq i \leq s} \log P(w_{t+j} \mid w_t).$$

Skip-Gram: A Concrete Example

Sentence. “the dog jumped over the fence”

Indexing.

$w_1 = \text{the}$, $w_2 = \text{dog}$, $w_3 = \text{jumped}$, $w_4 = \text{over}$, $w_5 = \text{the}$, $w_6 = \text{fence}$.

Window size $s = 2$, **center** $w_2 = \text{dog}$. Context words are w_1, w_3, w_4 .

Training pairs (center, context):

Pair	Use vectors	Score
(dog, the)	$\mathbf{v}_{\text{dog}}, \mathbf{v}'_{\text{the}}$	$\mathbf{v}'_{\text{the}}{}^T \mathbf{v}_{\text{dog}}$
(dog, jumped)	$\mathbf{v}_{\text{dog}}, \mathbf{v}'_{\text{jumped}}$	$\mathbf{v}'_{\text{jumped}}{}^T \mathbf{v}_{\text{dog}}$
(dog, over)	$\mathbf{v}_{\text{dog}}, \mathbf{v}'_{\text{over}}$	$\mathbf{v}'_{\text{over}}{}^T \mathbf{v}_{\text{dog}}$

Then slide the window: $w_3 = \text{jumped}$ becomes the new center, and we generate new pairs $(\text{jumped}, w_1), (\text{jumped}, w_2), (\text{jumped}, w_4), (\text{jumped}, w_5)$.

This pair-generation step is the entire training data preparation for word2vec.

Why Two Vectors Per Word?

Each word w gets two embeddings:

- $\mathbf{v}_w \in \mathbb{R}^D$: used when w plays the role of **input** / **center**.
- $\mathbf{v}'_w \in \mathbb{R}^D$: used when w plays the role of **output** / **context**.

Score for the pair (w_I, w_O) :

$$\text{score}(w_I, w_O) = \mathbf{v}'_{w_O}{}^\top \mathbf{v}_{w_I}.$$

Example. For pair (dog, jumped):

$$\text{score} = \mathbf{v}'_{\text{jumped}}{}^\top \mathbf{v}_{\text{dog}}.$$

Why two?

- Separates the two roles cleanly; the math becomes asymmetric in the right way.
- Empirically gives better embeddings than tying $\mathbf{v}_w = \mathbf{v}'_w$.

Final embedding. After training, most people keep $\mathbf{v}_w$ (or sometimes $\frac{1}{2}(\mathbf{v}_w + \mathbf{v}'_w)$).

Implementation: two `nn.Embedding` layers of size $W \times D$ each.

The Softmax Denominator Problem

The probability:

$$P(w_O \mid w_I) = \frac{\exp(\mathbf{v}'_{w_O}^\top \mathbf{v}_{w_I})}{\underbrace{\sum_{w \in S} \exp(\mathbf{v}'_w{}^\top \mathbf{v}_{w_I})}_{Z(w_I)}}$$

Bad news: computing the denominator requires summing over the entire vocabulary – $|V| \sim 10^5$ or 10^6 words.

Worse: we do this for every training example, every gradient step.

Two solutions in the original paper:

1. **Negative sampling** – approximate the softmax by sampling. (The modern standard.)
2. Hierarchical softmax – replace the W -way softmax by a binary tree of depth $\log_2 W$.

We treat them in that order. But first, let's make the softmax concrete with numbers.

Softmax: A Numerical Example

Tiny vocabulary $S = \{\text{dog, jumped, pizza}\}$, $W = 3$, $D = 2$.

Input embedding of the center word:

$$\mathbf{v}_{\text{dog}} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

Output embeddings of every possible context:

$$\mathbf{v}'_{\text{dog}} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad \mathbf{v}'_{\text{jumped}} = \begin{bmatrix} 2 \\ 1 \end{bmatrix}, \quad \mathbf{v}'_{\text{pizza}} = \begin{bmatrix} -1 \\ 0 \end{bmatrix}.$$

Compute the three scores:

$$\begin{aligned} \text{score}(\text{dog}) &= \mathbf{v}'_{\text{dog}}{}^{\top} \mathbf{v}_{\text{dog}} = 0 \cdot 1 + 1 \cdot 2 = 2, \\ \text{score}(\text{jumped}) &= \mathbf{v}'_{\text{jumped}}{}^{\top} \mathbf{v}_{\text{dog}} = 2 \cdot 1 + 1 \cdot 2 = 4, \\ \text{score}(\text{pizza}) &= \mathbf{v}'_{\text{pizza}}{}^{\top} \mathbf{v}_{\text{dog}} = -1 \cdot 1 + 0 \cdot 2 = -1. \end{aligned}$$

Softmax Numerical Example: The Probabilities

Exponentiate and normalize:

$$e^2 \approx 7.39,$$

$$e^4 \approx 54.60,$$

$$e^{-1} \approx 0.37.$$

$$Z = 7.39 + 54.60 + 0.37 \approx 62.36.$$

Softmax probabilities:

$$P(\text{jumped} \mid \text{dog}) = \frac{54.60}{62.36} \approx 0.875.$$

$$P(\text{dog} \mid \text{dog}) = \frac{7.39}{62.36} \approx 0.118, \quad P(\text{pizza} \mid \text{dog}) = \frac{0.37}{62.36} \approx 0.006.$$

Interpretation

The model thinks “*jumped*” is the likely context of “*dog*” because the dot product $\mathbf{v}'_{\text{jumped}} \mathbf{v}_{\text{dog}}$ is large.

Training pushes the score of the *actual* observed context up, and competing scores down.

Training Word2vec by Gradient Descent

Negative log-likelihood:

$$\min_{\mathbf{v}, \mathbf{v}'} \underbrace{\sum_{t=1}^T \sum_{\substack{-s \leq j \leq s \\ j \neq 0}}}_{\text{all (center, context) pairs}} \left[-\log P(w_{t+j} \mid w_t) \right].$$

Reading the loss:

- Observed pair (w_t, w_{t+j}) should have *high* probability $\rightarrow$ *low* $-\log P$.
- Unobserved (random) pairs implicitly carry weight through the denominator.

What gradient descent does:

1. Increases $\mathbf{v}'_{w_{t+j}}{}^\top \mathbf{v}_{w_t}$ for real (center, context) pairs.
2. Decreases $\mathbf{v}'_w{}^\top \mathbf{v}_{w_t}$ for all other vocabulary words w .

Initialization. Random; SGD or Adam with mini-batches over pairs.

Why Full Softmax Is Expensive

Big-O recap. “Cost is $O(f(n))$ ” means the work grows like $f(n)$ as n grows, ignoring constants.

The softmax denominator:

$$Z(w_I) = \sum_{w=1}^W \exp(\mathbf{v}'_w{}^\top \mathbf{v}_{w_I}).$$

Cost analysis:

- W dot products.
- Each dot product touches D coordinates.
- Total: $O(WD)$ per training pair.

Concrete example. $W = 100,000$, $D = 300$:

$$W \cdot D = 30,000,000 \text{ coordinate operations per pair.}$$

With billions of training pairs in a corpus, this is unworkable.

Note: a careful implementation computes *all* $P(w \mid w_I)$ in one pass over the vocabulary – $O(WD)$, not $O(W^2D)$. Naive code can accidentally do the W^2 version.

Gradient of the Log-Softmax

Goal: compute $\nabla_{\mathbf{v}_{w_I}} \log P(w_O | w_I)$ for SGD.

Start from:

$$\log P(w_O | w_I) = \mathbf{v}'_{w_O}{}^\top \mathbf{v}_{w_I} - \log Z(w_I)$$

where $Z(w_I) = \sum_w \exp(\mathbf{v}'_w{}^\top \mathbf{v}_{w_I})$.

Derivative of the first term:

$$\nabla_{\mathbf{v}_{w_I}} (\mathbf{v}'_{w_O}{}^\top \mathbf{v}_{w_I}) = \mathbf{v}'_{w_O}.$$

Derivative of the log-partition (chain rule):

$$\nabla_{\mathbf{v}_{w_I}} \log Z(w_I) = \frac{1}{Z} \sum_w \exp(\mathbf{v}'_w{}^\top \mathbf{v}_{w_I}) \mathbf{v}'_w = \sum_w P(w | w_I) \mathbf{v}'_w.$$

Combining:

$$\nabla_{\mathbf{v}_{w_I}} \log P(w_O | w_I) = \mathbf{v}'_{w_O} - \sum_{w=1}^W P(w | w_I) \mathbf{v}'_w$$

Move $\mathbf{v}_{w_I}$ toward the true context $\mathbf{v}'_{w_O}$ and away from the model's average prediction.

Negative Sampling

Idea: replace the softmax classification (“which word among W ?”) with many binary classifications (“is this pair real or fake?”).

For each observed pair (w_I, w_O) :

- Treat (w_I, w_O) as a **positive** example: real co-occurrence.
- Sample k words $\tilde{w}_1, \dots, \tilde{w}_k$ from a noise distribution q .
- Treat $(w_I, \tilde{w}_i)$ as **negative** examples: random.

Loss for one (w_I, w_O) **pair:**

$$\mathcal{L} = -\log \sigma(\mathbf{v}'_{w_O}{}^\top \mathbf{v}_{w_I}) - \sum_{i=1}^k \log \sigma(-\mathbf{v}'_{\tilde{w}_i}{}^\top \mathbf{v}_{w_I})$$

where σ is the sigmoid.

Per step cost: $O(kD)$ instead of $O(WD)$. Typically $k = 5$ to 20 .

This is the actual word2vec training procedure used in practice.

Why Hierarchical Softmax?

Regular softmax asks one big question:

“Among all W vocabulary words, which is the context?”

Cost: $O(WD)$.

Hierarchical softmax replaces it with a sequence of binary questions:

“Go left or right?” $\rightarrow$ “Go left or right?” $\rightarrow \dots$

until reaching the target leaf.

Setup.

- Build a *binary tree* whose **leaves** are the W vocabulary words.
- Each **internal node** (including the root) has its own embedding vector.

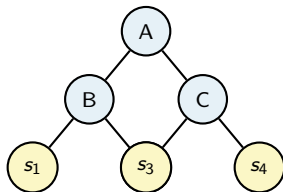
Internal nodes are *not* words – they are decision points.

Cost per training pair: $O(LD)$ where L is the path length to the target leaf. For a balanced tree, $L = \log_2 W$.

For $W = 10^5$: $\log_2 W \approx 17$. We replace 100,000 dot products with 17.

Hierarchical Softmax: The Math

Small example tree:



Each internal node has a vector $\mathbf{a}_{\text{node}} \in \mathbb{R}^D$ (learned).

At each node, branching probability uses a sigmoid:

$$P(\text{left} \mid w_l) = \sigma(\mathbf{a}_{\text{node}}^\top \mathbf{v}_{w_l}), \quad P(\text{right} \mid w_l) = 1 - \sigma(\cdot).$$

Probability of a leaf = product along its path. Example:

$$P(s_3 \mid w_l) = P(\text{right at A}) \cdot P(\text{left at C}).$$

Training: same gradient descent objective. The path probabilities replace the softmax denominator.

Hierarchical Softmax: Cost Comparison

Method	Cost per pair	$W = 10^4, D = 100$	$W = 10^5, D = 300$
Full softmax	$O(WD)$	10^6	$3 \cdot 10^7$
Hierarchical softmax	$O(\log_2(W) \cdot D)$	$\approx 1,300$	$\approx 5,100$
Negative sampling	$O(kD)$	$\approx 1,000$	$\approx 3,000$

Takeaways:

- Both tricks scale; full softmax does not.
- Negative sampling and hierarchical softmax are comparable in cost.
- Modern systems prefer negative sampling: simpler, more parallel, fewer hyperparameters.

Hierarchical softmax still matters when: you need exact probabilities (negative sampling only approximates), or you're working on tasks where the tree structure is meaningful (e.g., the WordNet hierarchy).

Why Huffman Trees?

Observation: cost is proportional to *path length* to the target leaf.

If a word appears **often**, its leaf is visited often. We want it to have a **short path**.

Huffman coding (1952): given word frequencies, build the tree that minimizes the *expected path length*

$$\mathbb{E}[L] = \sum_w f_w \cdot L_w$$

where f_w is the frequency of word w and L_w is its path length.

Two contrasting cases:

- **All words equally frequent** $\rightarrow$ balanced tree is optimal; every leaf has depth $\log_2 W$.
- **Very skewed frequencies** (typical for language: Zipf's law) $\rightarrow$ Huffman tree is much better.

For word2vec, English Wikipedia has Zipfian frequencies. Huffman gives big speedups.

Huffman Construction: How It Works

Algorithm.

1. Treat each word as a leaf with weight = frequency.
2. Find the two smallest-weight nodes; merge them into a new node whose weight is the sum.
3. Repeat until one node (the root) remains.

Worked example. Suppose we have these word frequencies:

$$f_{\text{do}} = 18, \quad f_{\text{you}} = 4, \quad f_{\text{know}} = 7, \quad f_{\text{the}} = 20,$$

$$f_{\text{way}} = 9, \quad f_{\text{of}} = 4, \quad f_{\text{devil}} = 5, \quad f_{\text{queen}} = 6.$$

Merging steps (always combine the two smallest):

1. $\text{you}(4) + \text{of}(4) = 8$
2. $\text{devil}(5) + \text{queen}(6) = 11$
3. $\text{know}(7) + 8 = 15$
4. $\text{way}(9) + 11 = 20$
5. $15 + \text{do}(18) = 33$
6. $\text{the}(20) + 20 = 40$

Huffman Example: Expected Path Length

Resulting Huffman path lengths (one valid assignment):

Word	Frequency f_w	Huffman path length L_w	$f_w \cdot L_w$
do	18	2	36
the	20	2	40
know	7	3	21
way	9	3	27
you	4	4	16
of	4	4	16
devil	5	4	20
queen	6	4	24
Total	73		200

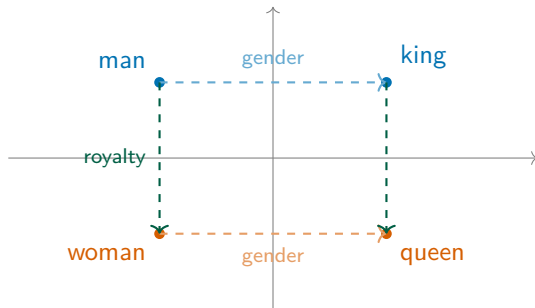
Expected Huffman path length: $\mathbb{E}[L]_{\text{Huffman}} = 200/73 \approx 2.74$.

Balanced binary tree with 8 words: every leaf at depth 3, so $\mathbb{E}[L]_{\text{balanced}} = 3$.

Vector Arithmetic in Embedding Space

The famous result:

$$\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}} + \mathbf{v}_{\text{woman}} \approx \mathbf{v}_{\text{queen}}$$



Geometric reading: “gender” and “royalty” are nearly orthogonal directions, so the parallelogram closes.

This is not magic. It is a direct consequence of the linear structure of the PMI matrix.

Examples From the Original Paper

Mikolov et al. (2013) found these analogies hold:

Relationship	Example 1	Example 2	Example 3
country : capital	France : Paris	Japan : Tokyo	Italy : Rome
big : bigger	small : larger	cold : colder	quick : quicker
country : currency	Czech : koruna	Vietnam : dong	Russia : rouble
country : sport	Japan : sumo	Brazil : soccer	USA : baseball
metal : symbol	copper : Cu	zinc : Zn	gold : Au
person : role	Einstein : scientist	Messi : footballer	Picasso : painter

In each row: the vector difference between columns 1, 2, 3 is approximately constant.

That is: $\mathbf{v}_{\text{France}} - \mathbf{v}_{\text{Paris}} \approx \mathbf{v}_{\text{Japan}} - \mathbf{v}_{\text{Tokyo}} \approx \mathbf{v}_{\text{Italy}} - \mathbf{v}_{\text{Rome}}$.

The embedding has learned that “capital of” is a linear operation in vector space.

Choosing the Dimension D

What should D be? (Some sources call it K – same thing.)

Extremes:

- $D = W$: the score matrix can be full-rank; the model almost memorizes word-pair interactions. No generalization.
- $D = 1$: not enough capacity to encode meaning.

What D controls:

- Smaller $D \rightarrow$ forces low-dim latent structure $\rightarrow$ better generalization across similar words.
- Larger $D \rightarrow$ more capacity, more risk of overfitting, more memory.

Parameter count. Two embedding matrices, each $W \times D$:

$$\text{params} = 2 WD.$$

For $W = 50,000$ and $D = 300$: $2 \cdot 50,000 \cdot 300 = 30$ million parameters.

Practical values:

Setting	Typical D
Small / specialized corpus	50, 100

GloVe: A Different Recipe

GloVe (Pennington, Socher, Manning, EMNLP 2014).

Build a global co-occurrence matrix X_{ij} = number of times word j appears in the context of word i . Then minimize:

$$\mathcal{L} = \sum_{i,j} f(X_{ij}) (\mathbf{u}_i^\top \mathbf{v}_j + b_i + b_j - \log X_{ij})^2$$

where f is a weighting function downweighting rare pairs.

Differences from word2vec:

- Operates on global statistics, not streaming windows.
- Explicitly factorizes the log co-occurrence matrix.
- Closer in spirit to the Lecture 6 latent factor view.

In practice: word2vec and GloVe give very similar results. Either is fine; both are matrix factorization in disguise.

A Spanish Example

Train word2vec on Spanish Wikipedia + BCRP minutes. What clusters near “*inflación*”?

Anchor: “inflación”	Anchor: “BCRP”	Anchor: “sol”
deflación	banco central	nuevo sol
deflactor	directorio	tipo de cambio
IPC	encaje	PEN
expectativas inflacionarias	tasa de referencia	dólar
núcleo subyacente	política monetaria	moneda

Analogy test (after training):

$$\mathbf{v}_{\text{Perú}} - \mathbf{v}_{\text{Lima}} + \mathbf{v}_{\text{Chile}} \stackrel{?}{\approx} \mathbf{v}_{\text{Santiago}}.$$

For applied econ work, Spanish embeddings from your own corpus often outperform translated English ones – especially for local entity names and terminology.

Section Summary

What we built:

- Skip-gram trained by predicting context words from the center word.
- Negative sampling makes training tractable: $O(k)$ instead of $O(|V|)$.
- Levy-Goldberg: word2vec is factorization of the shifted PMI matrix.
- Vector arithmetic captures linguistic regularities.

Three takeaways:

1. Embeddings emerge from a co-occurrence prediction task.
2. The geometry is linear-algebraic, not magical.
3. For Spanish / local-context economics, train your own, not a translated English model.

Next: we apply this machinery beyond words – to firms, judges, central banks, countries.

Embeddings for Economics

Beyond words: firms, judges, countries

The Recipe

The general pattern for econ applications:

1. Identify a set of **entities** you want to study (words, firms, judges, FOMC speakers, countries, sectors).
2. Find a **co-occurrence signal**: which pairs of entities appear together, traded together, mentioned together, voted together.
3. Train an **embedding** of each entity in $\mathbb{R}^K$.
4. Use the embeddings as **features** for downstream econometrics: regression, clustering, matching, causal inference.

The payoff:

- Captures soft, non-linear similarities that hand-coded categories miss.
- Reduces a high-dimensional sparse signal to a dense feature.
- Pre-trained on more data than your econ sample provides.

The trick is to identify the right co-occurrence signal.

Words: Text-as-Data

Gentzkow, Kelly, Taddy (2019, *JEL*). “Text as data.”

Survey of text-as-data methods for economics. The canonical reference.

Hansen & McMahon (2016, *JIE*). “Shocking language.”

- Use embeddings of FOMC statements to measure monetary policy “surprise”.
- Show that the surprise component predicts macro responses.

Hansen, McMahon, Prat (2018, *QJE*). “Transparency and deliberation within the FOMC.”

- Topic models on FOMC transcripts.
- Show that transparency changed FOMC member behavior.

Pattern in all of these: embed central bank language, regress macro outcomes on the embedded features.

Judges: Ideology from Text

Ash, Chen, Naidu and follow-ups: *Judge embeddings from opinions.*

Setup:

- Each judge writes opinions – thousands of words per judge.
- Treat the judge as an entity, each opinion as context.
- Train an embedding such that “similar” judges have similar vectors.

Findings:

- First principal component of judge embeddings $\approx$ liberal–conservative.
- Second component captures economic vs social conservatism.
- Useful for IV designs: random assignment of judges + embedded ideology = exogenous shock.

Judges as entities, opinions as the co-occurrence signal, ideology as the dimension that emerges.

Firms: Hoberg & Phillips

Hoberg & Phillips (2010, *RFS*; 2016, *JPE*).

Idea: use the text of 10-K filings to embed each firm.

Procedure:

1. Extract product-description sections from every 10-K.
2. Compute pairwise text similarity between firms.
3. Build a similarity-based industry classification (TNIC).

Why it matters:

- Goes beyond SIC / NAICS codes, which are coarse and slow to update.
- Captures soft product-market boundaries: Apple and Samsung look similar in 10-K text even when SIC says otherwise.
- Used in hundreds of finance papers.

An embedding of firms based on their own self-descriptions.

Countries: Embeddings from Trade

Idea: treat each country as an entity, trade flows as co-occurrence.

Gravity model:

$$\log \text{Trade}_{ij} = \alpha_i + \beta_j + \mathbf{u}_i^\top \mathbf{v}_j + \text{controls}_{ij} + \varepsilon_{ij}$$

where α_i, β_j are exporter and importer fixed effects, and $\mathbf{u}_i, \mathbf{v}_j$ are learned embeddings.

What the embeddings reveal:

- Geographic clusters (Latin America, EU, ASEAN).
- Industry-mix similarity (oil exporters, manufacturing economies).
- Cultural / linguistic ties not captured by distance alone.

Same math as recommender systems – countries are users, trade partners are items.

Section Summary

Four application areas, one technique.

Entity	Co-occurrence signal	What emerges
Words	nearby in text	meaning / syntax
Judges	co-authored opinions, same topic	ideology
Firms	similar 10-K text	product-market boundaries
Countries	trade flows	geographic + cultural blocks

Methodological advice for econ research:

- Embeddings are *features*, not the final analysis.
- Always validate with an out-of-sample task that you care about.
- Be careful with downstream standard errors – embeddings have estimation uncertainty.

The methodology is the same; the creativity is in choosing the entity and the signal.

Deep Embeddings

When the embedding is a neural network

From Lookup to Computation

Two regimes for producing an embedding:

Regime 1: Lookup. The embedding is a row of a matrix.

$$\mathbf{u}_s = \mathbf{E}[s, :]$$

This is what `nn.Embedding` does. Pure parameter, no computation.

Regime 2: Computation. The embedding is the output of a network.

$$\mathbf{u}_s = \phi_{\Theta}(\mathbf{x}_s)$$

where $\mathbf{x}_s$ is some raw feature (image, text, audio) and ϕ is a neural network with weights Θ .

Why it matters:

- Lookup needs an ID; can't handle new entities.
- Computation generalizes: a new image just needs a forward pass.

This solves cold start, and it's how every modern multimodal system works.

The PyTorch View

Lookup embedding:

```
1 emb = nn.Embedding(num_words, K)
2 u = emb(word_id)           # K-dim vector
```

Deep / computed embedding:

```
1 encoder = nn.Sequential(
2     nn.Conv2d(3, 64, 3),      # raw image -> features
3     nn.ReLU(),
4     nn.AdaptiveAvgPool2d(1),
5     nn.Flatten(),
6     nn.Linear(64, K)         # final embedding dimension
7 )
8 u = encoder(image)          # K-dim vector
```

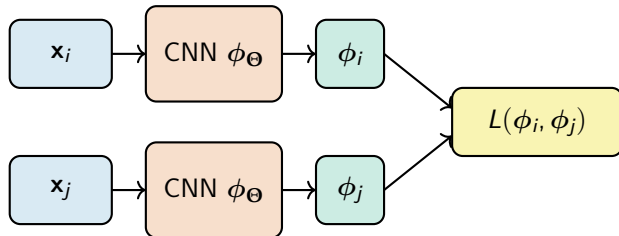
The output is the same shape – a K -dim vector. The difference is whether it's looked up or computed.

Same training tricks apply to both: similarity loss, negative sampling, etc.

Siamese Networks

Setup: train a network that maps any input to a useful embedding.

Same weights Θ !



Loss: small when (x_i, x_j) are similar, large when they aren't.

Key feature: the *same* network processes both inputs. Symmetric by construction.

Triplet Loss

Better idea than “compatible vs incompatible” pairs: use *triples*.

A triplet: (a, p, n) where

- a = anchor.
- p = positive (similar to anchor).
- n = negative (dissimilar to anchor).

Triplet loss:

$$\mathcal{L}(a, p, n) = \max(0, \|\phi_a - \phi_p\|^2 - \|\phi_a - \phi_n\|^2 + \alpha)$$

where $\alpha > 0$ is a margin.

Reading the formula: push the positive closer and the negative farther by at least α .

Where it's used:

- Face recognition (FaceNet, Schroff et al. 2015).
- Image search (Google Lens).
- Speaker recognition.

Hard-negative mining is essential – random negatives are usually too easy.

Visual Style: Veit et al. 2015

Goal: learn an embedding that captures *compatibility* between clothing items.

Training data:

- $\sim$ 1M Amazon products with category labels.
- Pairs “frequently bought together” = compatible.
- Random pairs across categories = incompatible.

Architecture: a Siamese CNN – two copies of the same conv net producing 128-dim embeddings.

Result: a learned embedding space where “items that go together” cluster, regardless of category.

A red T-shirt and a blue T-shirt are near each other; a red T-shirt and a leather belt are near each other; but a red T-shirt and a dress are far.

This is the canonical industry use case – it's how Amazon shopping recommendations work.

Modern Context: Contrastive Learning

Triplet loss generalizes to many forms of contrastive learning.

SimCLR, MoCo, DINO (2020s) for images:

- For each image, two augmented views = positive pair.
- Every other image in the batch = negative.
- Train the encoder so positives are close, negatives are far.

CLIP (Radford et al., 2021):

- Image-text dual encoder.
- Caption matches its image; other captions are negatives.
- The shared embedding space lets you do zero-shot classification.

SBERT (Reimers & Gurevych, 2019):

- Sentence-level Siamese BERT for semantic similarity.

All of these are direct descendants of the Siamese / triplet idea from the early 2010s.

Section Summary

From lookup to neural:

- Lookup embeddings: $\mathbf{u}_s = \mathbf{E}[s, :]$. Stored, not computed.
- Deep embeddings: $\mathbf{u}_s = \phi_{\Theta}(\mathbf{x}_s)$. Computed from features.

Training paradigms:

- Siamese networks: pair-based loss with shared weights.
- Triplet loss: anchor + positive + negative with a margin.
- Contrastive learning: many negatives per positive (SimCLR, CLIP, DINO).

All modern representation learning lives in this family.

Coming up: a critical limitation of everything we've seen so far – the embeddings are *static*.

Static vs Contextual Embeddings

The limit of word2vec

The Problem with Static Embeddings

Static embedding: each word has *one* vector, no matter the sentence.

The classic example:

- “I deposited my paycheck at the **bank**.”
- “The boat drifted to the river **bank**.”

Both occurrences of *bank* get *the same vector* from word2vec.

Worse, for econ:

- “The Fed hiked the **rate**.” (interest rate)
- “This is a high-**rate** of inflation.” (rate as ratio)
- “Rate the product 1–5 stars.” (rate as verb)

Three different meanings, one vector. Word2vec averages them all.

What We Want: Context-Dependent Vectors

The fix: make the embedding of a word depend on its surrounding sentence.

$$\mathbf{u}_{\text{bank}} \longrightarrow \mathbf{u}_{\text{bank}}(\text{context})$$

Two consequences:

- Different occurrences of the same word can have different vectors.
- The same word in different sentences can land in different parts of embedding space.

Three generations of solutions:

1. **ELMo** (2018): bidirectional LSTM produces context-dependent vectors.
2. **BERT** (2018): transformer encoder, contextual at every layer.
3. **GPT, LLaMA, modern LLMs**: causal transformers, contextual everywhere.

All three rely on the *attention* mechanism – the topic of next class.

What Attention Will Do for Us

Preview (full treatment next lecture):

For each word in a sentence, compute its embedding as a *weighted average* of *all other words'* embeddings.

The weights depend on how relevant each other word is.

Sneak peek: self-attention

$$\mathbf{u}_i = \sum_{j=1}^T \alpha_{ij} \mathbf{v}_j \quad \text{where} \quad \alpha_{ij} = \frac{\exp(\mathbf{q}_i^\top \mathbf{k}_j)}{\sum_l \exp(\mathbf{q}_i^\top \mathbf{k}_l)}$$

Each word i looks at every word j in the sentence with attention weight α_{ij} .

Result: the embedding of *bank* in “river bank” attends strongly to *river*; in “paycheck at the bank” it attends to *paycheck*.

Same word, different vectors – because the context is different.

Why It Matters for Economics

Static embeddings (word2vec, GloVe):

- Cheap to compute, easy to share.
- Useful for clustering or simple regression features.
- Limitation: polysemy + lack of context.

Contextual embeddings (BERT, GPT, modern):

- Same word can mean different things in different sentences.
- Capture fine-grained semantics: “**tasa** de interés” vs “**tasa** de mortalidad”.
- Required for nuanced sentiment, stance detection, narrative analysis.

For your work:

- Use sentence-transformers (SBERT, Spanish models like beto) over word2vec for FOMC / BCRP analysis.
- Treat word2vec as a baseline; modern contextual models give noticeably better accuracy on sentiment and topic tasks.

We'll see how this works under the hood next class.

Summary & Next Steps

Key Takeaways

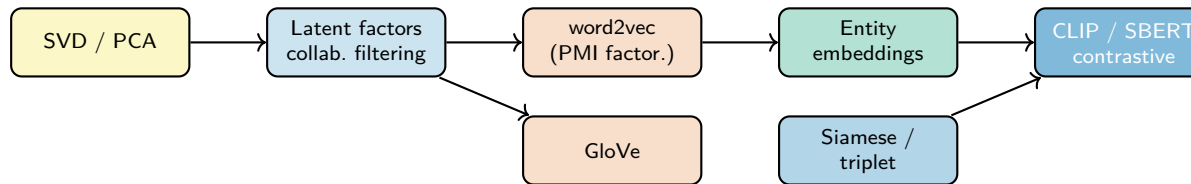
Technical highlights of today:

1. **Embeddings** represent objects as vectors where geometry captures useful relationships.
2. **LLE** preserves local linear reconstruction weights from high dim to low dim, via the matrix $\mathbf{M} = (\mathbf{I} - \mathbf{W})^\top (\mathbf{I} - \mathbf{W})$.
3. **Word2vec** learns embeddings by predicting context words from center words; objective is the average log-likelihood over a sliding window.
4. **Skip-gram softmax** costs $O(WD)$ per training pair – prohibitive at $W \sim 10^5$.
5. **Hierarchical softmax** replaces one W -way decision with a path of binary decisions: $O(\log_2 W \cdot D)$ per pair.
6. **Huffman trees** make frequent words cheaper by assigning them shorter paths – expected length below balanced for Zipfian data.
7. **Negative sampling** is the modern alternative: $O(kD)$ per pair, simpler to implement, default in practice.

Today's Big Ideas

1. **Embedding** = learned vector representation of a discrete object.
2. **Two geometries:** distance encodes clustering; inner product encodes direction (and is exactly a latent factor model).
3. **Word2vec** = skip-gram + negative sampling. Levy-Goldberg: it's matrix factorization of shifted PMI.
4. **Vector arithmetic** captures linguistic regularities – not magic, just linear structure.
5. **For economics:** embed firms, judges, countries, central bank text. Use as features for downstream analysis.
6. **Deep embeddings:** replace lookup with a neural network. Siamese, triplet, contrastive – all variants of the same idea.
7. **Static** → **contextual** is the next frontier. Attention is the mechanism.

The Big Picture



Reading the diagram:

- Yellow = Lecture 6 (linear factor methods).
- Red = today's content.
- Blue / white = where we're going.

Each box is the previous box's loss function applied to a new type of data, or a new architecture replacing the linear lookup.

Practical Recipes

When to use what:

- Need a quick embedding from your own corpus → **word2vec** or **GloVe**.
- Need state-of-the-art Spanish embeddings → **BETO**, **RoBERTa-es**, **sentence-transformers**.
- Embedding new entities (firms, countries, judges) → **custom skip-gram** on your co-occurrence signal.
- Image / multimodal embeddings → **CLIP**, **DINOv2**, **BLIP**.
- Visualization → **t-SNE** (small data), **UMAP** (large), **PCA** (reproducible).

Common pitfalls:

- Don't use t-SNE / UMAP coordinates as features – they're for visualization only.
- Don't trust word2vec analogies blindly; check sample size and corpus quality.
- Don't forget that embeddings are estimated; downstream standard errors should account for that.

Next Lecture: Attention and Self-Attention

Lecture 8:

- The attention mechanism: queries, keys, values.
- Self-attention: a sentence attends to itself.
- Multi-head attention: multiple “views” of the same sequence.
- Foundation for the Transformer (Lecture 9).

The transition: we move from *static* lookup tables (today) to *dynamic* context-dependent embeddings (next class).

Recommended reading:

- Mikolov et al. (2013), *Distributed representations of words and phrases*.
- Levy & Goldberg (2014), *Neural word embedding as implicit matrix factorization*.
- Gentzkow, Kelly & Taddy (2019), *Text as data*, JEL.
- Vaswani et al. (2017), *Attention is all you need* (prep for next class).

Questions?

References

- Roweis, S., & Saul, L. (2000). Nonlinear dimensionality reduction by locally linear embedding. *Science*, 290.
- van der Maaten, L., & Hinton, G. (2008). Visualizing data using t-SNE. *JMLR*, 9.
- McInnes, L., Healy, J., & Melville, J. (2018). UMAP: uniform manifold approximation and projection.
- Mikolov, T., Sutskever, I., Chen, K., Corrado, G., & Dean, J. (2013). Distributed representations of words and phrases. *NIPS*.
- Pennington, J., Socher, R., & Manning, C. (2014). GloVe: global vectors for word representation. *EMNLP*.
- Levy, O., & Goldberg, Y. (2014). Neural word embedding as implicit matrix factorization. *NIPS*.
- Schroff, F., Kalenichenko, D., & Philbin, J. (2015). FaceNet: a unified embedding for face recognition. *CVPR*.
- Veit, A., Kovacs, B., Bell, S., McAuley, J., Bala, K., & Belongie, S. (2015). Learning visual clothing style with heterogeneous dyadic co-occurrences. *ICCV*.
- Radford, A., et al. (2021). Learning transferable visual models from natural language supervision (CLIP). *ICML*.
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: sentence embeddings using Siamese BERT. *EMNLP*.
- Hansen, S., & McMahon, M. (2016). Shocking language: understanding the macro effects of central bank communication. *JIE*.
- Hoberg, G., & Phillips, G. (2016). Text-based network industries and endogenous product differentiation.